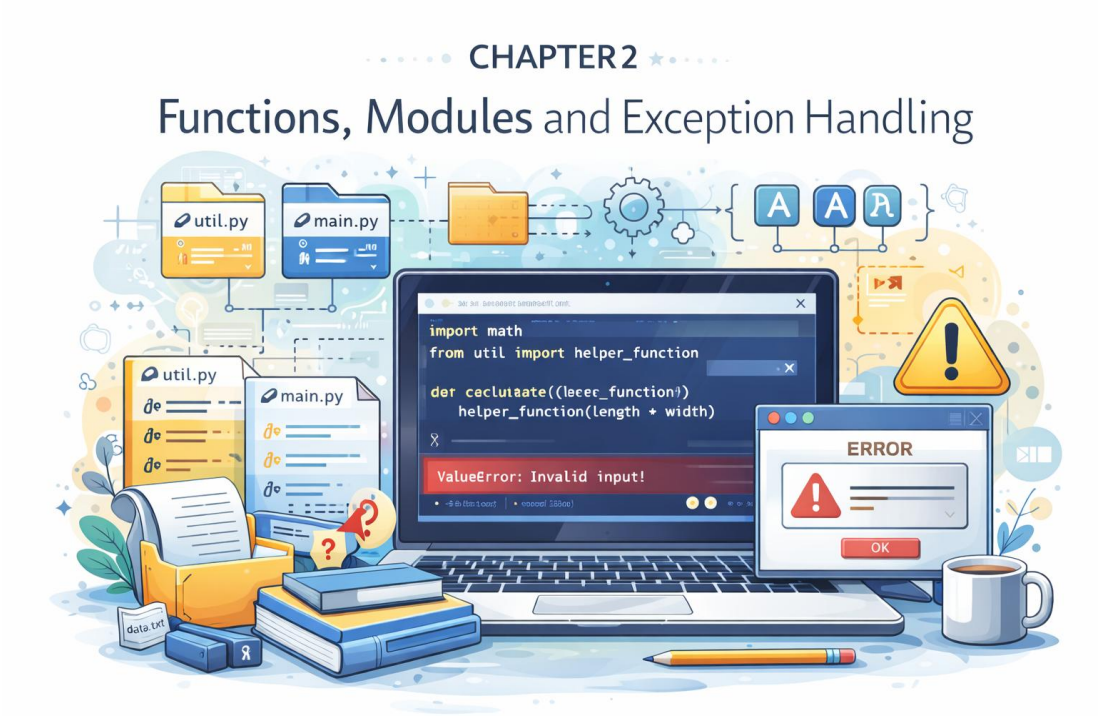


The following was generated using ChatGPT and further edited by Lucas Hartman

Chapter 2 – Functions, Modules, and Exception Handling

Functions, modules, and exception handling form the backbone of writing clean, maintainable Python programs. While Chapter 1 reviewed the essentials of input, output, conditionals, loops and some functions, this chapter takes you a step further by showing how to organize your code into meaningful pieces.

Functions allow you to break large problems into smaller, reusable units that make your programs easier to read, test, and expand. We will dive deep into more specific function details here. Modules let you place related functions and data into their own files, separating the logic of different "layers" of your program—such as keeping user interaction separate from data storage. Together, functions and modules encourage better design, making your programs more reliable and professional.

But even well-structured programs can run into unexpected problems. That's where exception handling comes in. Python provides a rich system for dealing with errors, allowing your programs to anticipate problems and respond gracefully instead of crashing. Whether it's validating user input, checking if a file exists, or handling unexpected results, exceptions give you control over what happens when things go wrong. By the end of this chapter, you'll not only know how to write and organize functions across multiple modules, but also how to protect your programs from errors using Python's try/except system. These three skills—functions, modules, and exception handling—are essential stepping stones toward building robust applications.

1. Functions – Deep Dive

Functions make code reusable and maintainable. They allow you to group logic into blocks that can be called multiple times. Let us explore parameters, arguments, returning values, default values, pitfalls with mutables, and variable scope.

1.1 Parameters vs Arguments

Parameters are the variables you declare in the function definition. Arguments are the values you pass in when calling the function.

You must pass arguments to a function in the order they are defined, or you can use parameters as the keyword and assign in any order/position. Typically, we see positional used overall.

```
def label(name, role):
    print(f"{name} — {role}")

label("Amira", "Admin")              # positional
label(role="Admin", name="Amira")    # keyword
```

1.2 Default Parameters

You can assign default values to parameters, making them optional when calling the function.

```
def welcome(user="Guest", excited=False):
    message = f"Welcome, {user}"
    if excited:
        message += "!"
    return message

print(welcome())
print(welcome("Alex", excited=True))
```

1.3 Reference vs assignment values

Take into consideration what type of variables you have within function parameters/arguments. Variables that are numbers, strings, tuples are all immutable(can't change) and get passed to functions via assignment. In a sense here the values are copied over to the function.

Variables that are lists, dictionaries, custom objects are mutable (able to change) and get passed via reference. That means when you go in and start to change up the object (like add to a list, sort a list, etc.) changes the original object as well. Sometimes you will want to do this which is fine. But just consider the functionality you are making in your functions. Typically you won't need to return the list, dictionary or object back as you are manipulating it there.

1.4 Mutable Default Pitfall

Avoid using mutable objects like lists or dictionaries as default parameter values. They can cause unexpected behavior.

```
def add_item(item, bucket=None):
    if bucket is None:
        bucket = []
    bucket.append(item)
    return bucket
```

1.5 Returning Values

Functions can return values using the return statement.

```
def add(a, b):
    return a + b

adding = add(4, 3)
```

You can also return early from a function or return multiple values at once as a tuple.

```
def find_first_even(nums):
    for n in nums:
        if n % 2 == 0:
            return n
    return None

def min_max(values):
    return min(values), max(values)

find_first_even([5, 3, 4, 2, 8])  # finds '4' as first even
low, high = min_max([3, 9, 2, 8]) # low as 2, high as 9
```

1.6 Return Void Values

Some methods do not return any value and just do some operation defined. These are called void functions

1.7 Defining Types

1.7.1 Setting Required Parameter Types

Python doesn't enforce strict data types but you can type-hints in our syntax with parameters. This ensures consistency and clarity on how the function should be used. To create this type hint we use the colon and list the type required.

If a wrong type is passed (e.g., a list instead of int), Python still runs – but the IDEs will highlight it.

```
def greet(name : str, age : int):
    print(f"Hello {name}, you are {age} years old.")
```

1.7.2 Defining Function Return Types

Just like with parameter types you can indicate what is the expected return type of a function. Again the Python language doesn't enforce but it will help with expectations of the code and allow the IDE to catch it.

To define a return type we use the arrow "`->`" then list the type between the function definition and colon.

```
def calculate_total(price : float, tax : float) -> float:
    return price + (price * tax)
```

1.7.3 Advanced Types

You can use `list[data type]` and `dict[keyType, datatype]` to denote lists/dictionaries for types (Python 3.10+)

You can also do Unions on types by using the vertical bar " | ". It acts as an "or" for types. So, we can define multiple types.

```
def add(a : int | float, b : int | float) -> int | float:
    return price + (price * tax)
```

This function now works with both adding two integers, one integer and one float, or two floats and returns matching data.

There is also sometimes where you want to have Optional returns. For example maybe we return a value or "None." For this we use our Union and list None as a type.

```
def find_user(name : str) -> str | None:
    ...
```

1.8 Local vs Global Variables

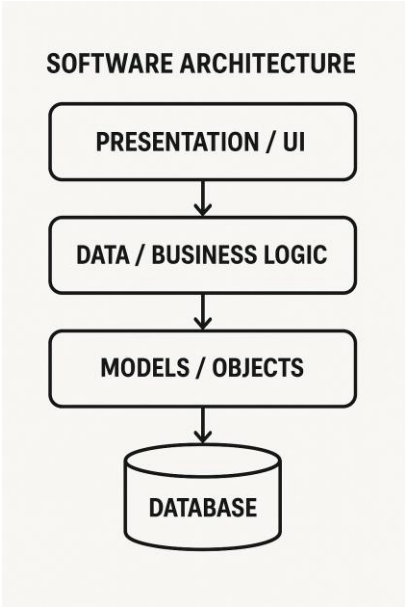
Variables declared inside a function are local to that function. Variables declared outside functions exist in the global scope. Functions should usually return values instead of modifying globals.

```
count = 0

def increment():
    count = 0
    count += 1
    print("local count:", count)

increment()
print("global count:", count)

def incremented(value):
    return value + 1

count = incremented(count)
print("global count:", count)
```

2. Modules – Separating Responsibilities

When we talk about modules, we are talking about the python(.py) file itself. From the get-go, you have been making modules.

Further utilization of these helps us make more efficient well-organized code. Modules allow us to separate different layers of code. One example which we will be doing is the UI layer with data layer. This way when we decide to update our UI layer by going graphical/web based, our data layer (and any other modules) will work with little to no modification needed.



2.1 Example database.py

Here is a minimal database.py file with save and find functions. It represents the data layer.

Notice that we are doing all the operations here and returning the variable/object. In this database layer we should NOT be printing or doing any output. Essentially the outputs here are returns from functions.

If you want to make sure things are happening correctly you may consider return True/False values and check those function calls if done correctly, then print out messages (in the UI layer of course).

```
# database.py
users = []

def save_user(user):
    users.append(user)

def find_user(username):
    for u in users:
        if u.get("username") == username:
            return u
    return None
```

2.2 Using Modules in main.py

You can import and use the functions defined in database.py from main.py.

```
import database

database.save_user({"username": "ana", "full_name": "Ana Patel"})
print(database.find_user("ana"))
```

You can also import all functions in a module, import specific functions, or change the name of the module importing to help in your coding. Importing all functions eliminates the need to reference the module name. Use the wildcard (*) that imports all.

```
from database import *

save_user({"username": "ana", "full_name": "Ana Patel"})
print(find_user("ana"))
```

Importing certain functions allows you to make your code base smaller if you don't need all the functions. Use the 'from' operator to tell what module then 'import' to signify the functions for importing.

```
from database import save_user

save_user({"username": "ana", "full_name": "Ana Patel"})
print(database.find_user("ana")) # didn't import this function
```

Changing the name of the functions/modules can eliminate collision with functions/variables you make in your modules. Use the 'as' operator and set the name.

```
import database as db

db.save_user({"username": "ana", "full_name": "Ana Patel"})
print(db.find_user("ana"))
```

2.3 Modules with mains()

There may be times when you want to test the functionality in your modules or that you want them to be able to run on their own. For this you would add a main() function, and call it. But when you import a module into another module, python looks at all the code and if you have a simple call to the main() function that code will run on import. This is why we need to use an if statement checking the file __name__ attribute to see if it is the __main__ file being ran or not. We discussed this briefly in Chapter 1.

```
# math_tools.py

def square(num):
    return num * num

def cube(num):
    return num * num * num

def main():
    value = int(input("Enter a number: "))
    print("Square:", square(value))
    print("Cube:", cube(value))

# Only runs if this file is executed directly
if __name__ == "__main__":
    main()
```

3. Built-in Modules

Python has a lot of built in modules. Some of these are already installed, others may need to be installed on your system.

- **pip** is the package installer that comes with python. It installs packages in the Python Package Index maintained by the developers of Python.
- To install you simply go to the terminal (on your machine or in PyCharm for virtual environments) and type "pip" then the package you would like to install

Why don't we just have the full system available from the get-go? If we include all of the libraries then our programs would be rather large for having the module files. If we aren't using them, we don't need them, so only import what you need.

The following are some of the well-known and useful modules you may come across. We will also look at others especially dealing with different file types in the coming modules.

Some popular modules include:

- math
- random
- os
- datetime

3.1 math

The math module provides advanced math functions beyond the basic operators. It also gives us the constant values of pi and e.

- **`sqrt(x)`** – returns square root of a number
- **`factorial(x)`** – returns factorial num * num-1, * num-2…. * 0
- **`gcd(a, b)`** - returns greatest commond divisor of a and b
- **`isclose(a, b, offset)`** – returns true if a and b are close based on offset. Good for checking float numbers as how computers store decimal numbers can be off especially when doing math. For example you could take 3/2 and calculates 1.5, but sometimes a computer computes it as 1.50000001 and when doing an equality check it will be false.
- **`hypot(a, b)`** – calculates the hypotenuse given the a and b of a right triangle

```
import math

print(math.sqrt(49))
print(math.factorial(5))
print(math.gcd(36, 60))
print(math.isclose(0.1+0.2, 0.3))
print(math.hypot(3, 4))
```

More on math here

3.2 random

The random module provides functions for generating random numbers and selections.

- **`random()`** – returns a random float number between 0 and 1.
- **`uniform(a, b)`** – returns a random float number between 'a' and 'b' values.
- **`randrange(a, b, c)`** – returns a random number between 'a' and 'b' using 'c' as a step value.
- **`randint(a, b)`** – returns a random number between 'a' and 'b' values.
- **`choice(sequence)`** – returns a random element from the given sequence.
- **`choices(sequence, weight, returned_length)`** - returns a list with a random selection from the given sequence. You can give weight of choosing and length of list being returned
- **`sample(sequence, length)`** – returns a list that contains 'length' defined elements from the sequence.
- **`shuffle(sequence)`** – randomly shuffles items in the list

```
import random

random.seed(42)
print(random.random())
print(random.uniform(10, 20))
print(random.randrange(0, 10, 2))
print(random.randint(1, 6))

items = ["A", "B", "C", "D"]
print(random.choice(items))
print(random.choices(items, weights=[5, 1, 1, 1], k=4))
print(random.sample(items, k=2))

nums = [1, 2, 3, 4, 5]
random.shuffle(nums)
print(nums)
```

More on random here

3.3 os

The os module allows interaction with the operating system in a cross-platform way.

- **os.name** is the current operating system name
- **getcwd()** – returns current working directory
- **path.join()** – automatically uses the correct path separator for the OS it's running on ("/" for Unix based and "\" for Windows)
- **path.exists(path)** – used to determine if a specified 'path' exists in the file system
- **path.isdir(dir)** – used to determine if a specified 'dir' exists in the file system
- **path.isfile(path)** – used to determine if a given 'path' refers to an existing regular file.
- **makedirs(dir)** – used to make a directory(folder) on the operating system at the given 'dir'. Optional parameter of **exist_ok** is default to **False**. If directory already exists then an exception is thrown. If set to **True** no exception thrown and function simply proceeds without creating the directory again.

```
import os

print(os.name)
print(os.getcwd())

path = os.path.join("data", "users.csv")
print(path)

print(os.path.exists("data"))
print(os.path.isdir("data"))
print(os.path.isfile(path))

os.makedirs(os.path.join("data", "archive"), exist_ok=True)
print(os.listdir("data"))
```

More on os here

3.4 datetime

The datetime module helps create, manipulate, and format dates and times.

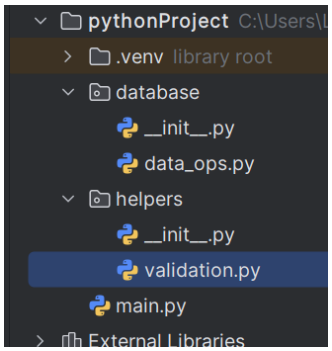
- **`datetime.now()`** – gets the current date and time from the operating system
- **`date.today()`** – gets the current date from the operating system
- **`datetime(year, month, day, hours, minutes, seconds)`** – creates a datetime object with the parameter values sent
- **`date(year, month, day)`** – creates a date object with the parameter values sent
- **`timedelta(days=#, seconds=#, minutes=#, hours=#, weeks=#)`** – used to perform arithmetic operations with datetime and date objects. Use keyword arguments to assign parameters
- **`strftime("%Y-%m-%d %H:%M")`** – used to format a datetime object into a string according to specified format code. Example above in parenthesis.
- **`strptime(stamp, "%Y-%m-%d %H:%M")`** – used to parse a string representation of a date and time into a datetime object. Takes on a string format of time stamp then uses the given format to parse.
    - **`%Y`** – Year with century – 2025
    - **`%y`** – Year without century – 25
    - **`%m`** – Month as a zero-padded number – 01, 02…12
    - **`%b`** – Abbreviated month name – Jan, Feb, etc.
    - **`%B`** – Full month name
    - **`%d`** – Day of the month as a zero-padded number – 01, 02…31
    - **`%H`** – Hour in 24 hour format – 00 – 23
    - **`%I`** – Hour in 12 clock, zero padded – 01-12
    - **`%S`** – Second zero padded – 00-59
    - **`%a`** – abbreviated weekday name – Sun, Mon, etc.
    - **`%A`** – Full weekday name – Sunday, Monday, etc.

```
import datetime as dt

now = dt.datetime.now()
today = dt.date.today()
print(now, today)

meeting = dt.datetime(2025, 9, 12, 10, 30)
deadline = meeting + dt.timedelta(days=3, hours=2)
print(deadline)

print(now.strftime("%Y-%m-%d %H:%M"))
print(now.strftime("%b %d, %Y (%a)"))

stamp = "2025-09-10 14:07"
parsed = dt.datetime.strptime(stamp, "%Y-%m-%d %H:%M")
print(parsed)
```

More on datetime here

4. Packages

Another element to help us better organize our code is the use of packages. This goes for all programming languages. Typically, you will have multiple utility files, data handling files, and when we get to next week, files that contain objects. These should be organized into packages to help.




```
project/
 ├── main.py
 ├── database/
 │    ├── __init__.py
 │    └── data_ops.py
 └── helpers/
      ├── __init__.py
      └── validation.py
```

In short you can think of packages as a folder, so when we are importing them, you must include the name of the package folder by a dot and the name of the python module to import.

```
from database.data_ops import save_record
from helpers.validation import validate_input

def main():
     data = input("Enter name: ")
     if validate_input(data):
          save_record(data)
     else:
          print("Invalid input.")
```

When we create packages you may notice a `__init__.py` gets automatically created(in most IDEs). This file is what tells the IDE that the folder is a package. Whenever you import a package, Python executes the code in this python file.

This file is often used for setup code, such as initializing configuration data, importing submodules, or defining what symbols should be available when the package is imported. We aren't getting into major specifics here but if you were to have a package inside of another package you could write code that when importing the main package folder automatically loads all sub modules from sub packages.

5. Exception Handling

Exceptions let you handle errors gracefully. Components of our exception handling include `try`, `except`, `else` and `finally`.

The `else` block executes only if no exception is raised with the try block. It provides a way to execute code that relies on the successful completion of the try block, without potentially catching exceptions that might be raised by that "success" code itself. If the except block is executed, the else block is skipped.

The `finally` block always executes, regardless of whether an exception occurred in the try block or was handled by an except block. It is typically used for clean up operations that must happen in any scenario, such as closing flies, releasing resources, or ensuring a specific state is maintained.

Both the else and finally blocks are optional to a try/except.

```
try:
    value = int(input("Number: "))
except ValueError:
    print("Not a valid integer.")
except Exception as e: # e is a variable we can access the exception
    print("Unexpected error:", e)
else:
    print("Success")
finally:
    print("Cleanup always runs")
```

Some exceptions are due to our own coding. A NameError gets thrown when we save have a variable "`name`" but we try to use it as variable "`Name`". These are logical errors that can be easily fixed.

When coding applications we want to handle any and all exceptions that could occur. Sometimes we don't need try/except to do so as if statements can help to check values, other times they are required.

- User Input/Data Processing – casting variables, math operations, zero division
- Accessing Collections – index out of bounds, key errors with dictionaries
- File I/O – opening, reading and writing to files
- External libraries – libraries may throw exceptions that we want to catch

4.1 Designing Exception Handling

You don't need to have a try/except for each single line of code. You should combine multiple lines of code that would potentially throw an exception in a try/except block. You simply need to line up with each exception that can get thrown.

```
try:
    value1 = int(input("Number: "))
except ValueError:
    print("Not a valid integer.")

try:
    value2 = int(input("Number: "))
except ValueError:
    print("Not a valid integer.")
```

Though, the above code maybe be warranted if you want to separate each value of getting input to not repeat the entire code. Instead, you would combine both inputs to something like before.

```
try:
    value1 = int(input("Number: "))
    value2 = int(input("Number: "))
except ValueError:
    print("Not valid integers.")
```

Even better, to make the program even more user friendly, create a function and make it loop to continue to ask for a valid input. This way you can get individual inputs and show an error for each. Here we send a text prompt to use in an input statement, attempt to get the input, and if it works, return the value.

```
def get_int(prompt):
    while True:
        try:
            value = int(input(prompt))
            return value
        except ValueError:
            print("Not valid integers.")
```

Some Exceptions:

- **Exception** – base exception for all exceptions. Use this if you want to simply catch all
- **ValueError** – raised when there is a wrong value for data type – user input for text to int() or float casting
- **ZeroDivisionError** – raised when second operator in division is zero
- **IndexError** – raised when index in a sequence doesn't exist
- **KeyError** – raised when a key does not exist in a dictionary

If you don't know what exception gets thrown use the base exception or run your code, when the exception is thrown record down what the Traceback response is.

More exceptions here

4.2 with Statement

The with statement in Python provides a convenient way to manage resources, ensuring that setup and teardown actions are performed automatically, even if errors occur within the code block. Here we are opening a file to read in using a with. No need for finally or else block.

```
try:
    with open("data.txt", "r") as f:
        content = f.readlines()
except FileNotFoundError:
    print("data.txt missing.")
```

5. Combining Modules and Exceptions

We can further better our code by making modules to help for specific tasks. Like here we can have an input tools module. We could have multiple functions that get values, or even just cast values handling them throughout the program.

This would be a separate module from our console UI, as we may need to use it when swapping it out to a GUI.

5.1 input_tools.py

```
def get_int_in_range(prompt, low, high):
    while True:
        try:
            n = int(input(prompt))
            if not (low <= n <= high):
                raise ValueError(f"Enter between {lo} and {hi}.")
            return n
        except ValueError as e:
            print("Invalid:", e)
```

5.2 main.py

```
import input_tools as it

age = it.get_int_in_range("Age (1-120): ", 1, 120)
print("Age recorded:", age)
```